Approach Document: CT-200 QA Traceability Backend

Bassa Srilakshmi · bassasrilakshmi@gmail.com · AI Engineering Internship Assignment, Tri9T AI · 23 October 2026

Repository: https://github.com/BASSASRILAKSHMI/ct200-qa

This document covers the data model, how the PDF was parsed (and every irregularity in it), version matching and where it breaks, LLM prompt and output-validation design, staleness, the debugging history, and the decision log. Deeper write-ups for individual parts are linked inline: pdf_findings.md, versioning.md, llm_generation.md, staleness.md.

1. What was built

```
PDF (v1, v2, ...)
  │  parse: text layer (OCR fallback) → lines → headings/paragraphs/lists/tables → tree
  ▼
SQLite ── Document ─< DocumentVersion ─< NodeVersion >─ LogicalNode
  │                                          ▲
  │        Selection ─< SelectionItem ────────┘   (pinned to exact NodeVersion rows)
  ▼
LLM (Groq, OpenAI-compatible) → validate → retry with feedback
  ▼
MongoDB Atlas (or JSON files): generation = test cases + snapshot of sources + every attempt
  ▼
Staleness, computed on every read: source hash → quote still present? → severity
```

All seven requested parts are implemented: ingestion and tree, versioning, browse API, selection API, LLM generation, staleness, and retrieval. There are 128 automated tests, run on Python 3.10 and 3.13 and on PyMuPDF 1.26–2.0. `scripts/demo_flow.py` runs the full v1 → selection → generate → v2 → staleness flow against the live API.

Stack. FastAPI, Pydantic v2, SQLAlchemy 2 + SQLite, MongoDB (Atlas free tier) via pymongo, and PyMuPDF with Tesseract as the OCR fallback. Two choices go beyond the suggested stack:

- a **JSON-file generation store** is used automatically when `MONGODB_URI` is unset, so the project runs with zero setup and a demo cannot fail on network access. Both stores pass the same tests (Mongo via `mongomock`).
- **OCR is the fallback, not the primary extractor.** This is justified in §3.1.

2. Data model

Table / collection	Purpose	Key decisions
`documents`	one manual	unique name
`document_versions`	one ingested file	`(document_id, version_number)` unique; file SHA-256 (idempotent re-upload); parse warnings; diff summary vs previous
`logical_nodes`	a section's **identity** across versions	created when a section first appears
`node_versions`	a section's **content in one version**	heading, number, level, body, blocks (JSON), flags, `content_hash`, `parent_id` (same version), `previous_id` (same logical node, earlier version), change status, match method. **Insert-only**

`selections`, `selection_items`	named set of node versions	all from one version; fingerprint = hash of `(node_version_id, content_hash)` pairs
`generations` (Mongo)	one LLM run	test cases, `sources` snapshot `{node_id, logical_node_id, version, content_hash}`, every attempt with raw text and errors, request key

The two-ID design carries most of the requirements. Because a NodeVersion is never updated, anything that points at it (a selection, a generation) keeps resolving to the exact text it was made from after any number of re-ingestions. Because the LogicalNode links versions, "the same section, now" is a single query.

The content hash is SHA-256 of the **normalised body** (NFKC, non-breaking hyphens folded, whitespace collapsed; en dashes kept because `22–42` is a range). Headings are compared separately. This normalisation is what lets unchanged sections hash identically in v1 and v2 even though v2 moves them to other pages and encodes hyphens differently.

3. PDF parsing

3.1 Approach and why

Both PDFs have a **real text layer**: every character, font name, size and position is available exactly. The pipeline reads that layer with PyMuPDF and uses Tesseract OCR only for pages without one (a scanned page). It also runs OCR as an independent cross-check (`scripts/ocr_crosscheck.py`).

This was decided by measurement, not preference. Forcing OCR on the real manual:

- reproduced the **same hierarchy** (27/28 sections, same parents and flags), which validates the hierarchy logic independently of fonts;
- but misread `±3 mmHg` as `+3 mmHg` (an accuracy specification), "Class II" as "Class Il", and the list numbers `1.` as `akWD –`; normalised dashes inconsistently; and flattened both tables.

Each of those would make an unchanged section look "changed" in v2, and `±3 → +3` silently corrupts a medical spec. So the text layer is primary, OCR is the safety net, and the OCR path is kept working and tested (it uses a stricter text-only heading rule because it has no bold information).

3.2 Hierarchy reconstruction

1. **Extract** lines with font, size and position, as one stream across all pages (so page breaks cannot cut sections). Table regions come from `find_tables()` geometry; their cells are filled from our own lines.
2. **Classify** each line as title, heading, paragraph, list item, or table. A heading must be bold **and** at the left margin **and** start with a section number. Margin, body size and right edge are learned from the document, not hard-coded.
3. **Build the tree from section numbers.** The parent of `a.b.c.d` is the longest proper prefix that exists. Level is the number of components. Irregularities are kept in document order and **flagged on the node**, never silently fixed.
4. **Coverage check.** Every extracted line must belong to exactly one element; unassigned or doubly-claimed lines are reported, and ingestion refuses to store such a tree.

3.3 Irregularities found and how each is handled

Found by dumping every line with its font/size/position (`scripts/inspect_pdf.py`) before writing parser code. Each one has a test in `tests/test_parser_ct200.py`.

#	Irregularity	Handling
1	`3.2` is printed at body size (11 pt) like a level-4 heading	level from numbering, never font size
2	bold 11 pt is also used for table headers	heading = bold + margin + number; table regions excluded
3	`2.1.1.1` directly under `2.1` (no `2.1.1`)	parent = nearest existing ancestor; true level 4 kept; flag `missing_parent:2.1.1`; no phantom node
4	`3.4` appears before `3.3`	document order kept; `3.3` flagged `out_of_order:after_3.4`
5	a numbered list `1. Normal` … `5. Crisis` inside 3.3 looks like five chapters	regular weight + indented → list; wrapped continuation joined to its item
6	two sections titled "Error Codes" (4.2, 7.1)	identity is never the title; distinct ids, different parents, both flagged `duplicate_title`
7	chapter headings at a page bottom; paragraphs split mid-sentence across pages	one continuous stream; a paragraph continues if the sentence is unfinished or the last line was a full-width line
8	tables with multi-line cells (2.1, 4.2)	cell geometry + our own text; stored as header + rows
9	ligatures (fi, ff) and U+2011 hyphens split into separate spans	normalised for display and hashing
10	section numbers inside sentences ("see 2.1, 4.3")	headings must start a bold line at the margin
11	three-line document title	merged into the root node
12	figures/images: **none** in either PDF (0 image blocks; the vector drawings are table borders)	not applicable here, but an image is never skipped silently: it becomes a `figure` block in its section, a "Figure N…" line just below becomes its caption, it counts toward the section hash, and it is reported as a parse warning (tested on a generated PDF)

3.4 What the first implementation got wrong, and how each was found

Failure	How it was found	Change
table cells corrupted by PyMuPDF's own extractor: `1 .5 seconds`, `0 –299 mmHg`	comparing `Table.extract()` with the raw line dump	geometry only from PyMuPDF, cell text from our lines
3.1 split into two paragraphs at a page break (the sentence ended exactly at the page bottom)	manual review of the printed tree	"full last line" rule added to "unfinished sentence"
first chapter swallowed as the title when a PDF has no separate title	a synthetic unit test	a numbered heading is never the title
right-edge estimate collapsed on tiny inputs	a synthetic unit test	percentile index rounds up
table text duplicated on PyMuPDF 1.26–1.28 (development used 2.0)	the word-coverage test failed **on a second machine**; reproduced with one virtualenv per PyMuPDF version	reject nested 1-row candidate tables; a line belongs to one table; parser warns on any doubly-claimed line

3.5 Validation methods

- **Word coverage:** every word in the PDF appears exactly once in the tree, and no word appears that is not in the PDF. No hand-written expectations are needed, and it caught bug 5 above.
- **Independent OCR parse:** same hierarchy from pixels alone.
- **Cross-version hash stability:** 24 sections are byte-identical in meaning across v1/v2 and must hash identically; exactly 4 must not.
- **Targeted tests** for each irregularity, plus synthetic tests for cases the PDF does not contain (duplicate numbers, unnumbered headings, OCR-only input, tables split across pages).

4. Versioning

Re-ingesting only inserts rows. Sections are matched to the latest stored version in passes, strictest first: **exact** (number + title) → **title under the matched parent** (survives renumbering) → **same number, similar title** (reworded heading) → **identical non-empty body** (moved + renamed) → **similar body with a clear winner**. A pass only matches when the candidate is unique on both sides; anything ambiguous falls through to added/removed.

Why: hashes alone cannot identify sections (every empty chapter heading has the same hash); numbers alone turn renumbering into removed + added; titles alone confuse 4.2 and 7.1. And a missed match is visible (removed + added, test cases flagged), while a wrong match silently re-attaches old test cases to other text, so ambiguity is refused.

Result on the real manual: 24 unchanged, 4 modified (2.1.1.1, 3.2, 4.2, 4.3), 1 added (5.3).

Where it breaks: a section split in two (one half loses lineage), two sections merged (one shows as removed), same-title siblings renumbered and reworded together (lineage lost, tested), a heavy rewrite under the same number (treated as a different section), only the previous version is compared (a section deleted and later restored gets a new identity), and thresholds set by judgement. Details: versioning.md.

Policies: identical bytes to the latest version → no new version; identical to an older version → a new version (a revert is a real event, and it correctly makes old tests "current" again); the coverage check failed → refused; concurrent uploads → the loser gets HTTP 409 (§9).

5. API

Browse: `GET /sections?version=`, `GET /tree`, `GET /nodes/{id}` (text, blocks, hash, flags, parent, path, children), `GET /search?q=&field=&flag=` and `GET /nodes/{id}/changes` (history plus a word diff to latest, with changed numbers listed). Selections: `POST/GET /selections`. Generation: `POST /selections/{id}/generate`. Retrieval: `GET /generations/{id}`, `GET /selections/{id}/generations?status=`, `GET /nodes/{id}/generations`. All are documented in Swagger at `/docs`.

Selections are **version-pinned** because their items are NodeVersion ids. They must come from one version (mixing v1 and v2 would describe a manual that never existed; 422). Unknown ids return 404, a reused name 409, and `include_descendants` expands chapter headings (which have no text of their own) at creation time, so the selection never grows later.

6. LLM generation

Prompt (llm_generation.md): sections are sent as `<section node_id="..">` data blocks, which the system prompt says are source material, not instructions. The output contract is shown as an

example object. Each test case must cite `source_node_ids` and a `requirement_quote` copied word for word. JSON mode is on, and temperature is 0.2.

Structured-output validation, in four layers, applied to the whole batch:

1. complete (not cut off, not empty);
2. parses as a JSON object;
3. Pydantic schema (fields, enums, non-empty steps);
4. meaning: 3–5 cases, only node ids that were sent, the quote really appears in the cited section, distinct titles.

Only a markdown fence is repaired. Content is never repaired, trimmed or partially accepted.

Retries: transport problems (429 with `Retry-After`, 5xx, timeouts) are retried inside the client with backoff. Config problems (bad key, unknown model) fail fast. A rejected reply is sent back with the exact validator errors, up to 3 replies. If all fail, the generation is **stored as failed with every raw reply** (HTTP 502), so the failure can be audited, and it contains no test cases.

Warnings that do not reject: a selected section no case cites; and `unverified_values`, which are numbers in an expected result that its cited text never states (added after real output did exactly that; see §9 Q1).

Same selection submitted twice: the key is `hash(selection fingerprint, prompt version, model)`. If it already succeeded, the stored result is returned (`reused: true`, HTTP 200) and the LLM is not called. LLM output is non-deterministic, and "the test cases for this exact text" should have one answer. `force=true` regenerates and records `supersedes`; nothing is deleted. Failed results are never reused. Two different selections with the same pinned texts share the result, because the key is about text, not names.

7. Staleness

Computed on every read against the latest version (never stored, so it cannot go stale itself). Per test case:

- **source level:** content hash of the latest version of the same logical section;
- **quote level:** if a cited source changed, is the test's verbatim quote still in the current text? Gone → **stale**. Still there → **review**;
- **severity: high** if a cited section was removed or a value the test depends on was changed by the edit; **medium** for a wording change in the quote; **low** if only other text changed.

One-word change vs. changed threshold: both are stale when they touch the quoted requirement, but the threshold change is high and the wording change medium, so they are not treated the same and a reviewer can triage. Both are tested.

Real result (Groq's four test cases for 3.2 + 4.2, generated from v1, checked against v2): "40 mmHg increments" and "E3 within 2 seconds" are **stale / high** (`40 → 30`, `2 → 1.5`); "180 mmHg target" and "E1 aborts" are **review / low**. A section-level check would have flagged all four. Limits are listed in staleness.md.

8. Engineering process: every real problem, and how it was caught

Problem	Caught by	Fix
PyMuPDF table text corruption (`1 .5 s`)	comparing library output with the raw dump	own cell text

paragraph split at a page break	manual tree review	full-line rule
title swallowing chapter 1	synthetic test	numbered heading ≠ title
duplicated table text on PyMuPDF 1.26–1.28	coverage test on another machine	fragment filtering + doubly-claimed-line warning
Swagger UI sent `document=string`, creating a document named "string"	manual testing in Swagger	`document` moved from form field to query parameter; regression test
diff said no number removed for 3.2 (`40 → 30`, but the new text mentions "original 40")	reading the history output	numbers taken from changed spans only
diff reported `E1` removed and added for `E1–E5 → E1–E6`	same	numbers cancel within one replacement
a matcher test passed for the wrong reason (accepted two outcomes)	checking why a new test passed first time	assertion pinned to the exact behaviour
`llama-3.3-70b-versatile` returned `model_not_found`	`scripts/check_setup.py` on first real setup (Groq retired it on 2026-08-16)	default `openai/gpt-oss-120b`; setup check lists usable models
demo crashed at the staleness step (variable reused)	running the demo end to end	separate variable
real Groq test asserted "220, 260, then 300 mmHg" (manual max 299)	reading the first real output	`unverified_values` warning
concurrent uploads gave a raw HTTP 500	testing the decision-log claim with two threads	HTTP 409; test proven to fail on the old code
images would have been skipped without a trace	final cross-check against the assignment's list (figures and captions)	figure blocks with captions + warning

9. Decision log

1. What is most likely to silently give wrong results without erroring? How would you catch it?

The *content* of a generated test case: its steps and expected result. Everything around it is checked mechanically (JSON, schema, cited ids, a verbatim quote), so a wrong test case can pass every check and look authoritative. This happened on the first real run. A test quoted, correctly, "inflates in 40 mmHg increments up to a maximum of 299 mmHg", then expected the cuff to reach "220, 260, then **300 mmHg**". 300 contradicts the 299 maximum, and nothing errored.

Caught now, partly: every test case carries `unverified_values`, the numbers and codes in its expected result that its cited sections never state (here `220, 260, 300`), and the generation gets a warning. It warns instead of rejecting because derived values like 180 + 40 can be correct. Next steps: a second, independent check of each expected result against the cited text (a different model or rule-based checks of limits like "never above 299"), and human sign-off before a generated case enters a test plan.

A close second is staleness itself: a test can depend on a sentence it did not quote, and a change there only produces **review/low**, never **stale**.

2. Where did you choose simplicity over correctness, and what would break first in production?

- **Concurrency was assumed away.** Ingestion computes "latest + 1" with no lock; duplicate-generation protection is an in-process lock; SQLite allows one writer. While writing this

answer it was tested with two threads uploading at the same moment: the database constraints prevented any corruption, but the second upload failed with a raw 500 and was lost, and **which file became version 1 depended on timing** (v2's file won every run). That is now a clear 409 "retry", but ordering is still not guaranteed. **This would break first**: two people uploading, or running more than one server worker. The real fix is a database-level lock (Postgres `SELECT … FOR UPDATE` or an advisory lock) and an explicit "expected previous version" on upload.

- Staleness is recomputed on every read, and the JSON store scans every file. That's fine for hundreds of generations, slow for many thousands (Mongo indexes cover the queries already).
- Version matching only compares with the previous version (a deleted-then-restored section gets a new identity), and its thresholds (0.6 / 0.85) are judgement, not tuned on data.

3. Name one input you did not handle, and what the system does with it.

A **scanned (image-only) page containing a table**. The OCR fallback produces text lines but has no table geometry, so the table becomes a run of paragraphs with cells interleaved by reading order. What the system does: it ingests the page (no line is lost, so the coverage check passes), records the warning "OCR used for pages [n]: no font weight, heading detection is text-only", and returns it with the version. It does **not** reconstruct the table, and the staleness and quote checks on that section become weaker because cell text order is unreliable. Handling it properly would need OCR layout analysis (e.g. table structure recognition) rather than line-level OCR.

(Two other known gaps: a **two-column layout** would interleave columns in reading order, which the coverage check cannot see because every word is still present once; and an **unnumbered heading** is kept but nested under the current section with an `unnumbered_heading` flag, since its level cannot be known from text.)

10. What would be done differently with more time

1. **Verify expected results, not just quotes:** rule checks against limits stated in the cited text, plus an independent LLM judge, with disagreements sent to a human.
2. **Concurrency-safe ingestion:** Postgres with row locks, and an upload that names the version it expects to replace.
3. **Multiple quotes per test case**, so staleness can cover every sentence a test depends on.
4. **Match across all earlier versions**, not just the previous one, and tune matcher thresholds on a labelled set of real manual revisions.
5. **Store cross-references** ("see 2.1, 4.3") as edges, so a change in 4.3 can flag tests on 3.3.
6. **Table-aware OCR** for scanned pages.
7. A small **review UI**, which the brief excluded, because the staleness reasons are written to be read by a person.